TECHNICAL INTERNSHIP REPORT

AWS DevOps Engineer Internship Assignment

Documentation of Cloud Infrastructure & Web Deployment

Candidate Name:	Galla Harshitha
Email Address:	harshithagalladevops@gmail.com
GitHub Profile:	https://github.com/Harshitha-Galla5
Submission Date:	July 09, 2026

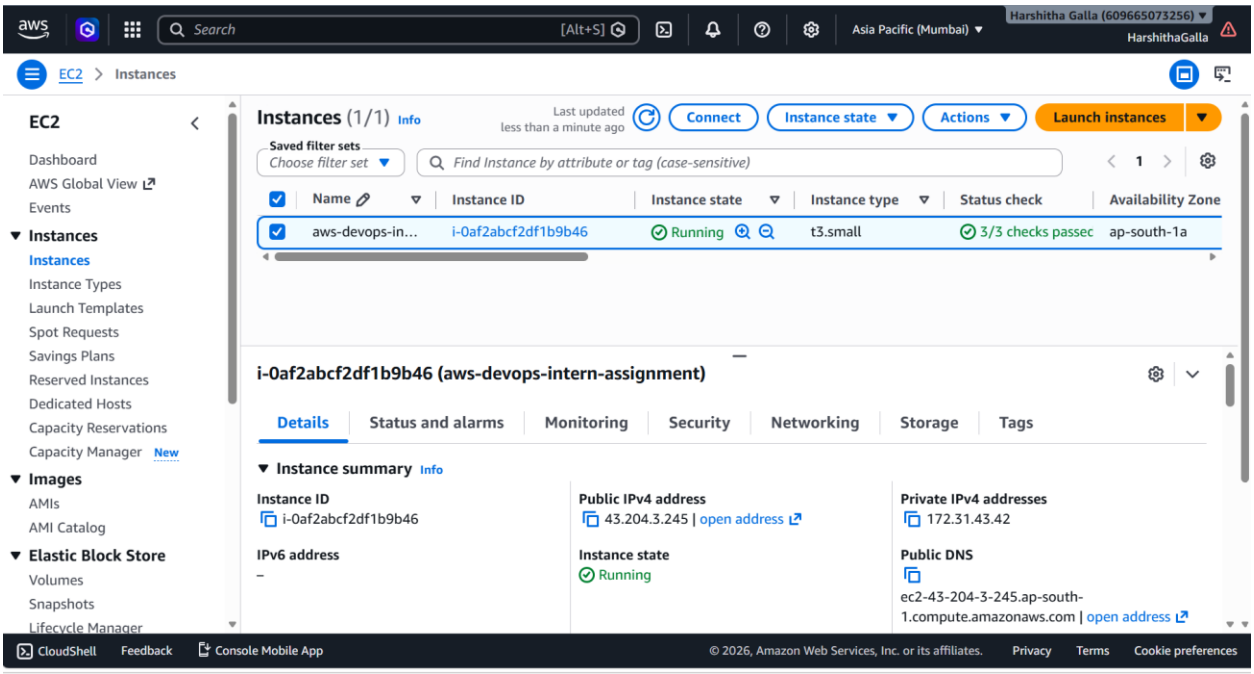
1. AWS Infrastructure Services Used

Amazon EC2 (Elastic Compute Cloud)

Amazon EC2 was utilized to launch a secure Ubuntu Linux virtual machine that served as the foundational hosting platform for this assignment. The EC2 node provided the cloud compute resources required to configure the underlying package sets, manage dependencies, manage server configurations, and expose the live static application via modern networking pipelines.

Key Architectural Roles:

- Hosted the production Ubuntu server instance
- Configured and served the custom static website framework
- Handled basic remote Linux administration and telemetry testing
- Provided isolation vectors via custom Nginx and Docker engine setups

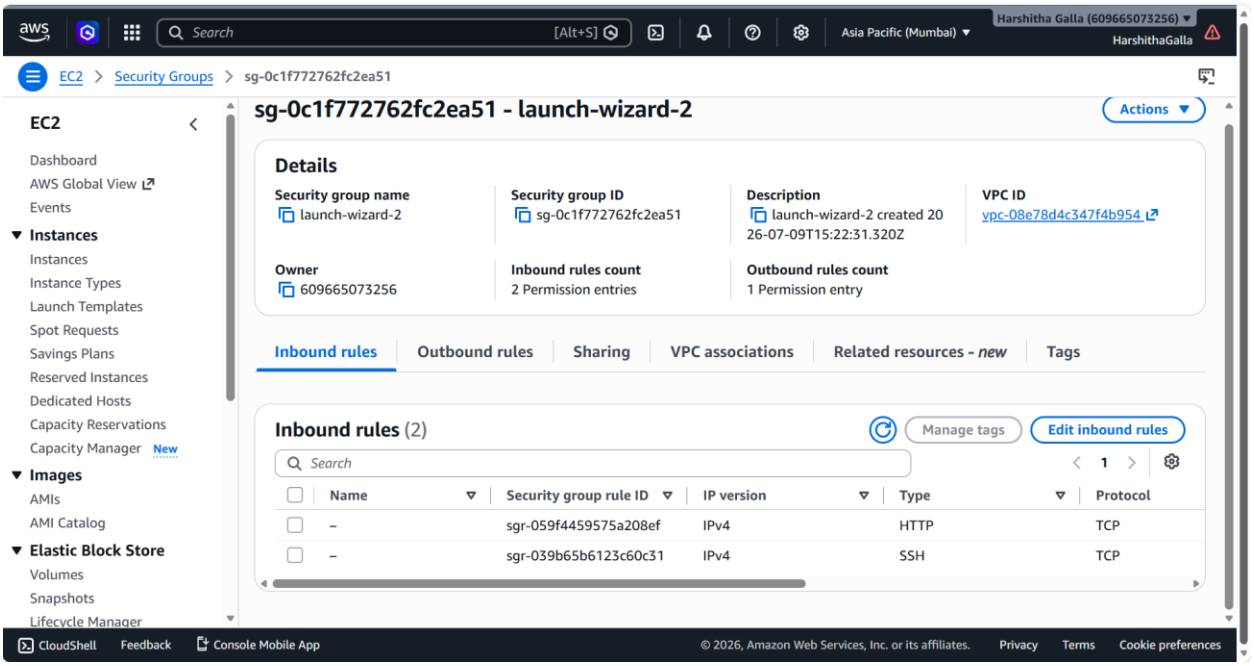


Security Group (Virtual Firewall)

A stateful Security Group was bound to the EC2 compute instance to act as a granular perimeter virtual firewall. This ruleset explicitly dropped unexpected ambient web scans while ensuring necessary application development channels remained transparently reachable.

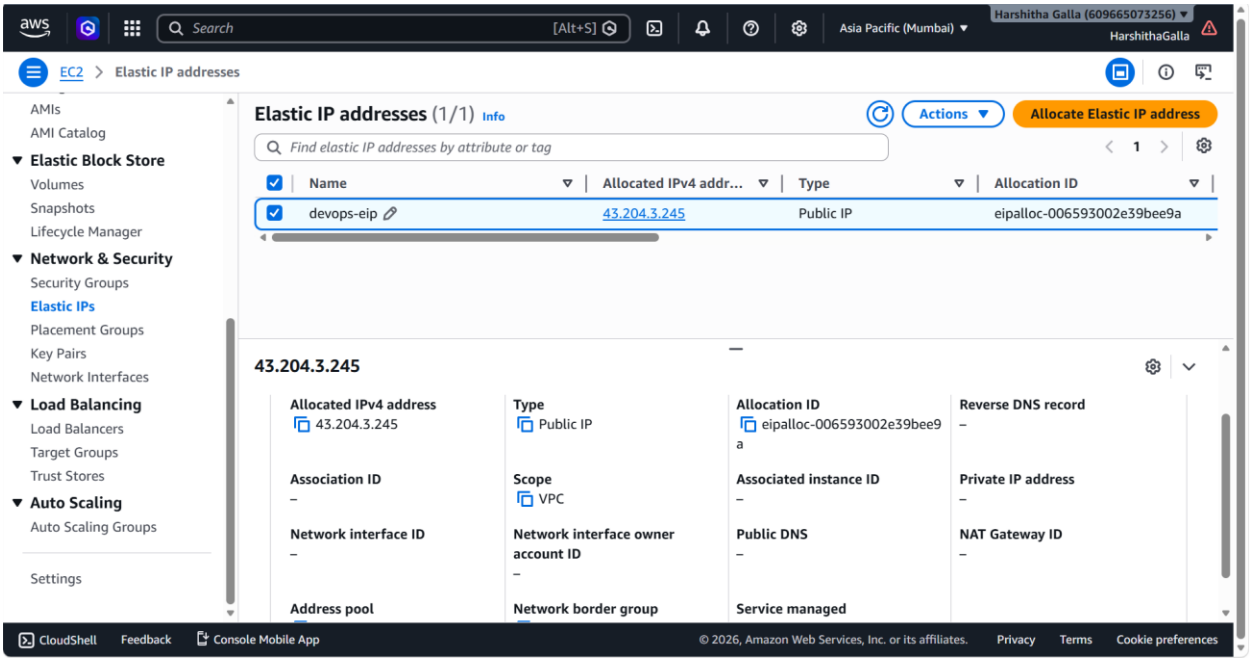
Configured Rule Standard:

Port Protocol	Inbound Rule Objective	Access Perimeter Scope
SSH (Port 22)	Secures encrypted interactive command-line access for remote development pipelines.	Restricted Administrative DevOps Scope
HTTP (Port 80)	Opens standard web routing access to handle global incoming site user requests.	Globally Public Access (0.0.0.0/0)



Elastic IP Address (Static Networking Bonus Task)

An Elastic IP (EIP) address was allocated inside the VPC environment and associated directly with our operational instance. Because traditional public cloud IPs cycle dynamically following compute reboots, utilizing a persistent Elastic IP secures a static, unyielding entry point that guarantees reliable domain resolutions and uninterrupted user accessibility.

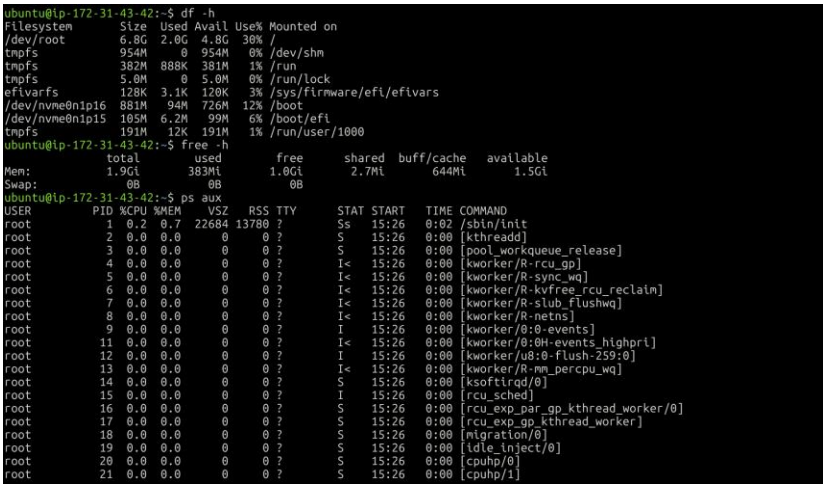

Search
[Alt+S]
Asia Pacific (Mumbai)
Harshitha Galla (609665073256)
HarshithaGalla
EC2 > Elastic IP addresses
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Trust Stores
Auto Scaling
Auto Scaling Groups
Settings
Elastic IP addresses (1/1) Info
Actions
Allocate Elastic IP address
Find elastic IP addresses by attribute or tag
Name
Allocated IPv4 addr...
Type
Allocation ID
devops-eip
43.204.3.245
Public IP
eipalloc-006593002e39bee9a
43.204.3.245
Allocated IPv4 address
43.204.3.245
Type
Public IP
Allocation ID
eipalloc-006593002e39bee9a
Reverse DNS record
–
Association ID
–
Scope
VPC
Associated instance ID
–
Private IP address
–
Network interface ID
–
Network interface owner account ID
–
Public DNS
–
NAT Gateway ID
–
Address pool
Network border group
Service managed
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

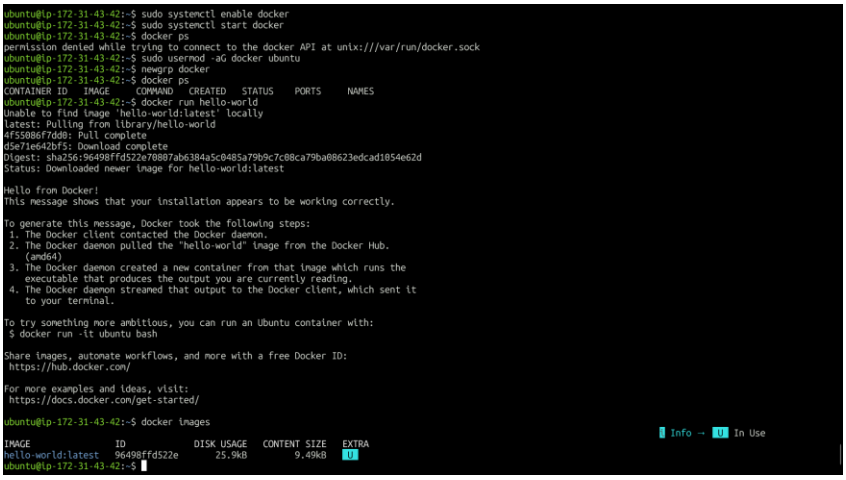
2. Linux System Administration & Implementation Shell Commands

The provisioning, lifecycle validation, and container platform deployment were executed via the standard Linux bash environment. The following implementation table details the sequential logic and execution purposes of the system utilities used:

Terminal Command Run	Technical Operational Objective & System Execution Meaning
`sudo apt update`	Synchronizes the local package database against source mirrors to fetch the absolute latest updates and patch sheets.
`sudo apt upgrade -y`	Upgrades all installed server package layers to newer structural dependencies without waiting for interactive prompts.
`sudo apt install nginx -y`	Fetches, structures, and launches the high-performance Nginx web server engine on the host system.
`nginx -v`	Exposes the active underlying compiled metadata version of Nginx to cross-reference security compliance benchmarks.
`sudo systemctl status nginx`	Queries the systemd process manager to explicitly verify that the Nginx daemon service is running stably.
`sudo systemctl restart nginx`	Flushes active runtime caches and forces an Nginx service cycle to instantly load new configurations and code assets.
`df -h`	Produces a clear, human-readable summary of mounted storage file volumes to track available storage allocations.
`free -h`	Exposes system RAM distribution parameters, breaking out active usage channels from buffered spaces.
`ps aux`	Produces an exhaustive, real-time snapshot summary of all active processes executing across the environment kernels.
`sudo apt install docker.io -y`	Installs the core upstream containerization infrastructure runtime software components directly on the node.
`sudo systemctl enable docker`	Registers the primary container runtime daemon to automatically run on instance boot.

`sudo systemctl start docker`	Fires up the main container hosting platform engine layers on the machine environment instantly.
`sudo docker run hello-world`	Pulls down and runs a minimalistic verification container asset to guarantee runtime operations are fully functional.





3. Practical Engineering Challenges Encountered & Resolutions

Real-world infrastructure configurations rarely execute cleanly without engineering adjustments. The structural friction points found during this deployment cycle were systematically analyzed and mitigated as follows:

- **Security Access Controls & Network Firewalls**

  Initially, while ports were being open, verifying explicit network routes was paramount to prevent accidental public blockades. Thorough end-to-end trace validations ensured SSH connections and internet web access routes mapped cleanly to the instance without introducing security vulnerabilities.

- **Nginx Path Mappings & Context Initialization**

  Replacing standard default placeholder files at /var/www/html/ required meticulous root ownership considerations. Elevating folder access through structured administrative command patterns cleanly unlocked deployment spaces without creating permanent permission vulnerabilities across the instance filesystem.

- **Web Daemon Initialization Validation**

  Ensuring standard service routing state metrics worked perfectly via systemd diagnostics prior to web lookups prevented silent configuration failures, ensuring that config syntax anomalies were caught before traffic hit the server.

- **Docker Core Engine Integration Mapping**

  Setting up upstream micro-container runtimes on modern kernels required precise service sequencing. Enabling and testing the container layer using sandbox images proved the setup's integrity before scaling up services.

4. Key Engineering Competencies & Technical Takeaways

This technical internship assignment provided highly impactful exposure to baseline enterprise systems mechanics and infrastructure-as-a-service (IaaS) principles:

- Mastered cloud resource lifecycle provisioning strategies within the Amazon EC2 dashboard framework.
- Established strong security boundaries using stateful firewall ingress/egress security policies.
- Aquired intermediate proficiency in Linux systems administration, folder handling, and diagnostics.
- Engineered production web hosting platforms using high-throughput Nginx reverse-proxy architectures.
- Implemented modern networking standards utilizing AWS Elastic IP blocks for immutable cloud accessibility.

- Explored containerization strategies by managing active system layers using the Docker container ecosystem.
- Maximized professional documentation standards by binding code state controls using Git and GitHub workflows.

5. Operational Engineering Time Tracking & Conclusion

Time Allocation Analysis

To measure delivery velocities accurately, the lifecycle implementation overhead across the 3 to 4 total hours expended is cataloged below:

Lifecycle Engineering Execution Phase	Time Invested
Launching & provisioning the baseline EC2 node environment	**30 Minutes**
Securing environment perimeters via custom Security Groups	**15 Minutes**
Configuring remote interactive sessions via public SSH protocols	**10 Minutes**
Compiling and fine-tuning the active Nginx web server engine	**45 Minutes**
Deploying and testing custom static web application assets	**30 Minutes**
Installing Docker system packages and confirming engine runtime logs	**30 Minutes**
Allocating, routing, and testing static AWS Elastic IP elements	**15 Minutes**
Documenting structures and synchronizing files up to GitHub repositories	**30 Minutes**
Conducting final network security reviews, performance health tests, and telemetry validations	**20 Minutes**

Assignment Conclusion

This execution exercise cleanly establishes an end-to-end framework for provisioning and managing high-availability web assets inside the Amazon Web Services enterprise cloud ecosystem. Implementing these steps provided valuable exposure to critical cloud architecture, network security design, system administration, and basic container platform mechanics.

By completing both core tasks and advanced bonus options—such as static network route bindings with Elastic IPs and localized container verification cycles with Docker—the overall deployment strategy demonstrates standard operational industry practices.